Assignment 1 — Characterising an AMOC time series

Course: Data Analysis for Physical Oceanography (MSc)
Weight: Assignment 1 of 2
Issued: 5 June 2026
Due: **19 June 2026** (submitted as a git repository)
Effort: ∼2 weeks

Aim

Take one Atlantic Meridional Overturning Circulation (AMOC) time series, get to know it, and describe it quantitatively — its basic time-domain statistics and its **spectrum**. You will reuse and extend the starter code from the spectra & filtering lecture, and submit a small, tested, reproducible analysis.

This is a "characterise a dataset properly" exercise. The marks are for doing a few standard things carefully and interpreting them honestly.

Learning outcomes

By the end you will be able to:

1. Load and clean a real oceanographic time series (gaps, sampling, units).
2. Summarise a series with appropriate statistics and explain what each tells you.
3. Estimate and read a power spectrum, and identify the dominant timescales.
4. Verify a variance budget with Parseval's theorem.
5. Package the analysis as tested code in a reproducible git repository.

The dataset

Use the **AMOCatlas** package to download an AMOC observing-array dataset of your choice:

```
pip install AMOCatlas
```

```
from amocatlas import read
ds = read.rapid()      # 26N  (also: read.osnap(), read.move(),
print(ds)              #        read.samba(), read.fw2015(), ...)
```

`read.<array>()` returns a standardised `xarray` dataset; explore it with `print(ds)` to see the variables, units, and time grid.

Choosing your series

- Pick **one** scalar time series (one variable, one location) to be the focus.
- Report its length, sampling, and processing honestly — that is part of the task.
- **Constraint:** if you use the RAPID 26°N `moc_transports` product, you may **not** use the headline overturning series (`moc_mar_hc10`) that we analysed in the lecture. Choose a different component instead (e.g. the Ekman, Gulf Stream / Florida Straits, or upper-mid-ocean transport).

State clearly at the top of your writeup which array, variable, and time span you chose, and why.

Required work

Part A — Characterise the series (time domain)

1. Report the record length, sampling interval, and the number/pattern of missing values, and state your gap-handling choice and why.
2. Compute and interpret the **mean, standard deviation, and range**, and show the **distribution** (a histogram is fine).

(The seasonal cycle, trends, and persistence/decorrelation are left for a later — you do not need them here.)

Part B — The spectrum (frequency domain)

1. Compute a **power spectrum** of your series. Choose a method that gives a readable, reliable estimate — Welch's overlapped-segment averaging is the natural choice. State your segment length and overlap.
2. Identify the **dominant timescales** (peaks) and describe the overall shape (red? white?).
3. Verify your variance budget with **Parseval**: the spectrum integrated over frequency should equal the variance of the series. Back this with a test (below).

Apply a filter (needed for the figures)

Low-pass filter your series by **convolving a window** — a Tukey window is a good default (`pandas rolling(..., win_type="tukey")`) — to isolate a timescale of interest. You will overlay the filtered series and its spectrum in the two required figures.

Recommended (for stronger marks, not required)

- **Part C — filter design:** justify your window and cutoff using its frequency response (does it pass what you want and suppress what you don't?), and compare against a plain boxcar of the same width.
- **Confidence intervals:** add a chi-squared confidence band to your spectrum and state the degrees of freedom.

What to implement (build on the starter package)

Work in the `code/spectra_filtering` package from the lecture:

- `analysis.summary_stats` — your mean (and the other basic statistics).
- `spectra.welch_psd` — your spectral estimate (use `parseval_ratio` to check it).
- `filters.tukey_lowpass` (provided) — the window filter for your figures.

Keep functions small and tested; the script/notebook that makes your figures should import them, not re-implement them. (The package also contains `seasonal_cycle` and `decorrelation_timescale` stubs — those are for later.)

Required tests

Your repository must include (at least) two passing `pytest` tests:

1. **The mean is computed correctly** (e.g. `summary_stats` returns the expected mean on a known input).
2. **Parseval is satisfied** — the integral of your spectrum equals the variance of the series to within a small tolerance.

Required figures

1. **The filtered series overlaid on the raw series** (time on the x-axis).
2. **The spectrum of the filtered series overlaid on the spectrum of the raw series** (log–log frequency axis). *Confidence intervals are recommended.*

Deliverables

A **git repository** containing:

1. The code: your implemented `summary_stats` and `welch_psd`, plus the script or notebook that reproduces your figures from the raw download.
2. A passing test suite (`pytest` runs green), including the two required tests.
3. The two required figures.
4. A short writeup (∼2 pages, `.rst` or `.md`) characterising your series and answering Parts A and B. Prose, not bullet points.

Submission & git workflow

Submit the URL of your repository and your report on the Moodle. Follow the solo-git workflow from the course: work on a branch, open a pull request into `main`, and **tag** the commit you want marked (`git tag assignment1 && git push -tags`). Your tests must run from a clean checkout (include a `requirements.txt`).

Assessment (rubric – see separate)

Strong attempts at the recommended parts (filter-design justification, confidence intervals) can lift a borderline grade. Marks reward *correct, honest, well-explained* analysis over volume.

Tips & common pitfalls

- **Detrend / de-mean before spectral estimation**, or the lowest frequencies will dominate and distort everything.
- **Interpolating gaps is itself a mild low-pass** — note it.
- **Mind your units and your frequency axis** (cycles per day? per year?); plot spectra on **log–log** axes.
- Tie what you see in the spectrum back to the timescales you expect physically.

Resources

- Lecture deck and starter code (`code/spectra_filtering`, `code/make_figures.py`).
- Emery & Thomson, *Data Analysis Methods in Physical Oceanography*, Ch. 5 (§5.6 *Spectral analysis*; §5.10 *Digital filters*).
- Percival & Walden (1993), *Spectral Analysis for Physical Applications*.
- AMOCatlas: https://github.com/AMOCcommunity/AMOCatlas
- J. M. Lilly, course notes on spectra & autocovariance (jmlilly.net).